

LAPORAN PRAKTIKUM
STRUKTUR DATA
“BINARY TREE AND GRAPH”



disusun Oleh:

NIDA TSABITA ARIBA

2511532025

Dosen Pengampu:

Dr. WAHYUDI, S.T., M.T.

Asisten Praktikum:

JOVANTRI IMMANUEL GULO

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Alhamdulillah rabbil ‘alamin, segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, nikmat, dan hidayah-Nya sehingga saya dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan judul “*BINARY TREE DAN GRAPH TRAVERSAL*”.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Struktur Data. Laporan ini diharapkan dapat memperdalam wawasan saya mengenai penerapan konsep struktur data non-linear, khususnya *Binary Tree* dan *Graph Traversal* dalam bahasa pemrograman Java, serta implementasinya dalam pengelolaan dan penelusuran data.

Saya menyampaikan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Struktur Data, serta asisten praktikum yang telah memberikan bimbingan dan arahan selama pelaksanaan kegiatan praktikum. Ucapan terima kasih juga saya sampaikan kepada teman-teman praktikan yang telah memberikan bantuan, dukungan, serta kesempatan berdiskusi dalam menyelesaikan tugas praktikum ini.

Saya menyadari bahwa laporan ini masih memiliki banyak kekurangan dan keterbatasan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum di masa mendatang. Meskipun demikian, saya berharap laporan ini dapat memberikan manfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	2
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN	4
2.1 Program <i>Node</i>	4
2.2 Program <i>BTree</i>	8
2.3 Program <i>BtreeDriver</i>	14
2.4 Program <i>GraphTraversal</i>	20
BAB III KESIMPULAN.....	28
3.1 Kesimpulan	28
DAFTAR PUSTAKA	30

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pembelajaran Struktur Data, mahasiswa tidak hanya mempelajari cara menyimpan data, tetapi juga mempelajari berbagai struktur dan algoritma yang digunakan untuk mengelola data secara efisien. Salah satu materi penting dalam struktur data adalah struktur data non-linear, seperti *Binary Tree* dan *Graph*, yang banyak digunakan untuk merepresentasikan hubungan antar data yang lebih kompleks.

Binary Tree merupakan struktur data berbentuk pohon yang terdiri atas *node* akar (*root*), *node* anak kiri (*left child*), dan *node* anak kanan (*right child*). Struktur ini banyak digunakan dalam berbagai aplikasi komputer karena mampu mempermudah proses penyimpanan, pencarian, dan pengelolaan data. Selain itu, *Binary Tree* juga memiliki beberapa metode *traversal*, yaitu *Preorder*, *Inorder*, dan *Postorder* yang digunakan untuk menelusuri seluruh *node* dalam *tree*.

Selain *Binary Tree*, terdapat pula struktur data *Graph* yang digunakan untuk merepresentasikan hubungan antar simpul (*vertex*) yang saling terhubung melalui sisi (*edge*). *Graph* banyak diterapkan pada berbagai bidang, seperti jaringan komputer, sistem navigasi, media sosial, dan pemetaan hubungan antar objek. Untuk melakukan penelusuran pada *graph* digunakan algoritma *traversal*, yaitu *Depth First Search (DFS)* dan *Breadth First Search (BFS)*.

Pada praktikum ini dilakukan implementasi *Binary Tree* melalui *class Node*, *BTree*, dan *BtreeDriver*, serta implementasi *Graph Traversal* menggunakan algoritma *DFS* dan *BFS*. Program dibuat menggunakan bahasa pemrograman Java untuk memahami cara membangun struktur data, menghubungkan node atau simpul, serta melakukan proses penelusuran data.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar *Binary Tree* dan *Graph*, mengetahui cara kerja *traversal* pada kedua struktur data tersebut, serta mampu mengimplementasikannya menggunakan bahasa pemrograman Java.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data *Binary Tree* dan *Graph*.
2. Memahami cara membentuk dan mengelola *node* pada *Binary Tree*.
3. Memahami proses *traversal* pada *Binary Tree* menggunakan metode *Preorder*, *Inorder*, dan *Postorder*.
4. Memahami konsep *Graph* serta representasinya menggunakan *adjacency list*.
5. Memahami cara kerja algoritma *Depth First Search (DFS)* dan *Breadth First Search (BFS)* pada *Graph*.
6. Mengimplementasikan *Binary Tree* dan *Graph Traversal* menggunakan bahasa pemrograman Java.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman mengenai struktur data non-linear, khususnya *Binary Tree* dan *Graph*.
2. Meningkatkan kemampuan dalam mengimplementasikan struktur data menggunakan bahasa pemrograman Java.
3. Memahami cara melakukan *traversal* pada *Binary Tree* dan *Graph*.
4. Melatih kemampuan dalam merancang hubungan antar *node* atau simpul pada suatu struktur data.
5. Membantu memahami penerapan *Binary Tree* dan *Graph* dalam penyelesaian berbagai permasalahan komputasi.

6. Meningkatkan kemampuan analisis dan pemecahan masalah melalui implementasi algoritma *traversal*.

BAB II

PEMBAHASAN

2.1 Program *Node*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan9_2511532025;

public class Node_2511532025 {
    int data_2025;
    Node_2511532025 left_2025;
    Node_2511532025 right_2025;
    public Node_2511532025(int data_2025) {
        this.data_2025 = data_2025;
        left_2025 = null;
        right_2025 = null;
    }
    public void setLeft_2025(Node_2511532025 node_2025) {
        if (left_2025 == null)
            left_2025 = node_2025;
    }
    public void setRight_2025(Node_2511532025 node_2025) {
        if (right_2025 == null)
            right_2025 = node_2025;
    }
    public Node_2511532025 getLeft_2025() {
        return left_2025;
    }
    public Node_2511532025 getRight_2025() {
        return right_2025;
    }
    public int getData_2025() {
        return data_2025;
    }
    public void setData_2025(int data_2025) {
        this.data_2025 = data_2025;
    }
    void printPreorder_2025(Node_2511532025 node_2025) {
        if (node_2025 == null)
            return;
        System.out.print(node_2025.data_2025 + " ");
        printPreorder_2025(node_2025.left_2025);
        printPreorder_2025(node_2025.right_2025);
    }
    void printPostorder_2025(Node_2511532025 node_2025) {
        if (node_2025 == null)
            return;
        printPostorder_2025(node_2025.left_2025);
        printPostorder_2025(node_2025.right_2025);
        System.out.print(node_2025.data_2025 + " ");
    }
    void printInorder_2025(Node_2511532025 node_2025) {
        if (node_2025 == null)
            return;
    }
}
```

```

        printInorder_2025(node_2025.left_2025);
        System.out.print(node_2025.data_2025 + " ");
        printInorder_2025(node_2025.right_2025);
    }
    public String print_2025() {
        return this.print_2025("", true, "");
    }
    public String print_2025(String prefix_2025, boolean
isTail_2025, String sb_2025) {
        if (right_2025 != null) {
            right_2025.print_2025(prefix_2025 +
(isTail_2025 ? "| ":"| "), false, sb_2025);
        }
        System.out.println(prefix_2025 + (isTail_2025 ? "\\-
- ":"/-- ") + data_2025);
        if (left_2025 != null) {
            left_2025.print_2025(prefix_2025 +
(isTail_2025 ? " ":"| "), true, sb_2025);
        }
        return sb_2025;
    }
}

```

Kode Program 2.1 Program *Node*

Kode di atas berfungsi untuk membuat sebuah *node* pada struktur data *Binary Tree* dalam bahasa Java. *Class Node_2511532025* digunakan sebagai komponen dasar pembentuk *Binary Tree*, di mana setiap *node* memiliki data, anak kiri (*left*), dan anak kanan (*right*).

Variabel data_2025 digunakan untuk menyimpan nilai pada *node*. *Variabel left_2025* digunakan untuk menunjuk *node* anak kiri, sedangkan *right_2025* digunakan untuk menunjuk *node* anak kanan. Jika *node* belum memiliki anak, maka kedua variabel tersebut bernilai *null*.

Class ini juga menyediakan *method* untuk menambah anak kiri dan anak kanan, mengambil data *node*, mengubah data *node*, serta melakukan *traversal Binary Tree* menggunakan metode *Preorder*, *Inorder*, dan *Postorder*. Selain itu terdapat *method print_2025()* yang digunakan untuk menampilkan struktur *tree* secara visual.

Melalui program ini dapat dipahami bahwa *node* merupakan elemen dasar pada *Binary Tree* yang saling terhubung membentuk struktur pohon dengan hubungan *parent* dan *child*.

2.1.1 Langkah Kerja Program *Node*

1. Membuat *class Node_2511532025*

```
public class Node_2511532025 {
```

Kode Program 2.2 *Class Node*

Class ini digunakan untuk merepresentasikan sebuah *node* pada struktur data *Binary Tree*.

2. Membuat atribut data dan anak *node*

```
int data_2025;  
Node_2511532025 left_2025;  
Node_2511532025 right_2025;
```

Kode Program 2.3 Atribut *Node*

Atribut ini digunakan untuk menyimpan data serta *node* anak kiri dan anak kanan.

3. Membuat *constructor*

```
public Node_2511532025(int data_2025) {  
    this.data_2025 = data_2025;  
    left_2025 = null;  
    right_2025 = null;  
}
```

Kode Program 2.4 *Constructor Node*

Constructor digunakan untuk menginisialisasi data *node* dan mengatur anak kiri serta anak kanan menjadi *null*.

4. Membuat *method setLeft_2025()* dan *setRight_2025()*

```
public void setLeft_2025(Node_2511532025 node_2025)  
{  
    if (left_2025 == null)
```

```

        left_2025 = node_2025;
    }
    public void setRight_2025(Node_2511532025 node_2025)
    {
        if (right_2025 == null)
            right_2025 = node_2025;
    }

```

Kode Program 2.5 *Method Set Node*

Method ini digunakan untuk menambahkan *node* anak kiri dan anak kanan.

5. Membuat *method* *get* dan *set* data

```

public Node_2511532025 getLeft_2025() {
    return left_2025;
}
public Node_2511532025 getRight_2025() {
    return right_2025;
}
public int getData_2025() {
    return data_2025;
}
public void setData_2025(int data_2025) {
    this.data_2025 = data_2025;
}

```

Kode Program 2.6 *Method Getter dan Setter*

Method ini digunakan untuk mengambil maupun mengubah data dan *node* anak.

6. Membuat *method* *traversal tree*

```

void printPreorder_2025(Node_2511532025 node_2025) {
    if (node_2025 == null)
        return;
    System.out.print(node_2025.data_2025 + " ");
    printPreorder_2025(node_2025.left_2025);
    printPreorder_2025(node_2025.right_2025);
}
void printPostorder_2025(Node_2511532025 node_2025)
{
    if (node_2025 == null)
        return;
    printPostorder_2025(node_2025.left_2025);
    printPostorder_2025(node_2025.right_2025);
    System.out.print(node_2025.data_2025 + " ");
}

```

```

void printInorder_2025(Node_2511532025 node_2025) {
    if (node_2025 == null)
        return;
    printInorder_2025(node_2025.left_2025);
    System.out.print(node_2025.data_2025 + " ");
    printInorder_2025(node_2025.right_2025);
}

```

Kode Program 2.7 *Method Traversal*

Method ini digunakan untuk menampilkan isi *Binary Tree* dengan urutan *Preorder*, *Inorder*, dan *Postorder*.

7. Membuat *method print_2025()*

```

public String print_2025() {
    return this.print_2025("", true, "");
}
public String print_2025(String prefix_2025, boolean
isTail_2025, String sb_2025) {
    if (right_2025 != null) {
        right_2025.print_2025(prefix_2025 +
(isTail_2025 ? "| ":" "), false, sb_2025);
    }
    System.out.println(prefix_2025 + (isTail_2025
? "\\-- ":"/-- ") + data_2025);
    if (left_2025 != null) {
        left_2025.print_2025(prefix_2025 +
(isTail_2025 ? " ":"| "), true, sb_2025);
    }
    return sb_2025;
}
}

```

Kode Program 2.8 *Method Print Tree*

Method ini digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk diagram sehingga hubungan antar *node* dapat terlihat dengan jelas.

2.2 Program *BTree*

Kode Program yang dipraktikkan adalah sebagai berikut:

```

package pekan9_2511532025;

public class BTree_2511532025 {
    private Node_2511532025 root_2025;
    private Node_2511532025 currentNode_2025;
    public BTree_2511532025() {

```

```

        root_2025 = null;
    }
    public boolean search(int data_2025) {
        return search_2025(root_2025, data_2025);
    }
    private boolean search_2025(Node_2511532025 node_2025, int
data_2025) {
        if (node_2025.getData_2025() == data_2025)
            return true;
        if (node_2025.getLeft_2025() != null)
            if (search_2025(node_2025.getLeft_2025(),
data_2025))
                return true;
        if (node_2025.getRight_2025() != null)
            if (search_2025(node_2025.getRight_2025(),
data_2025))
                return true;
        return false;
    }
    public void printInorder_2025() {
        root_2025.printInorder_2025(root_2025);
    }
    public void printPreorder_2025() {
        root_2025.printPreorder_2025(root_2025);
    }
    public void printPostorder_2025() {
        root_2025.printPostorder_2025(root_2025);
    }
    public Node_2511532025 getRoot_2025() {
        return root_2025;
    }
    public boolean isEmpty_2025() {
        return root_2025 == null;
    }
    public int countNodes_2025() {
        return countNodes_2025(root_2025);
    }
    private int countNodes_2025(Node_2511532025 node_2025) {
        int count_2025 = 1;
        if (node_2025 == null) {
            return 0;
        } else {
            count_2025 +=
countNodes_2025(node_2025.getLeft_2025());
            count_2025 +=
countNodes_2025(node_2025.getRight_2025());
            return count_2025;
        }
    }
    public void print_2025() {
        root_2025.print_2025();
    }
    public Node_2511532025 getCurrent_2025() {
        return currentNode_2025;
    }
    public void setCurrent_2025(Node_2511532025 node_2025) {
        this.currentNode_2025 = node_2025;
    }

```

```

    }
    public void setRoot_2025(Node_2511532025 root_2025) {
        this.root_2025 = root_2025;
    }
}

```

Kode Program 2.9 Program *QuickSort*

Kode di atas berfungsi untuk membuat struktur *Binary Tree* dalam bahasa *Java*. Class *BTree_2511532025* digunakan untuk mengelola *node-node* yang terdapat pada *Binary Tree*. Class ini memiliki atribut *root_2025* sebagai akar (*root*) *tree* dan *currentNode_2025* sebagai *node* yang sedang aktif digunakan.

Program menyediakan beberapa *method*, seperti pencarian data (*search_2025*), *traversal tree* (*printInorder_2025*, *printPreorder_2025*, dan *printPostorder_2025*), menghitung jumlah *node* (*countNodes_2025*), mengecek apakah *tree* kosong (*isEmpty_2025*), serta menampilkan struktur *tree* (*print_2025*).

Melalui program ini, *Binary Tree* dapat dikelola dengan lebih mudah karena seluruh operasi dasar pada *tree* telah disediakan dalam bentuk *method*.

2.2.1 Langkah Kerja Program *QuickSort*

1. Membuat class *BTree_2511532025*

```

public class BTree_2511532025 {

```

Kode Program 2.10 Class *BTree*

Class ini digunakan untuk mengelola struktur data *Binary Tree*.

2. Deklarasi variabel *root* dan *current node*

```

    private Node_2511532025 root_2025;
    private Node_2511532025 currentNode_2025;

```

Kode Program 2.11 Variabel *Root* dan *Current Node*

root_2025 digunakan untuk menyimpan *node* akar (*root*) *Binary Tree*. *currentNode_2025* digunakan untuk menyimpan *node* yang sedang aktif.

3. Membuat *constructor BTree*

```
public BTree_2511532025() {  
    root_2025 = null;  
}
```

Kode Program 2.12 *Constructor BTree*

Constructor digunakan untuk membuat *Binary Tree* baru dengan kondisi awal *root* bernilai *null*.

4. Membuat *method search()*

```
public boolean search(int data_2025) {  
    return search_2025(root_2025, data_2025);  
}
```

Kode Program 2.13 *Method Search*

Method ini digunakan untuk mencari data pada *Binary Tree*.

5. Membuat *method serach_2025()*

```
private boolean search_2025(Node_2511532025  
node_2025, int data_2025) {  
    if (node_2025.getData_2025() == data_2025)  
        return true;  
    if (node_2025.getLeft_2025() != null)  
        if  
(search_2025(node_2025.getLeft_2025(), data_2025))  
            return true;  
    if (node_2025.getRight_2025() != null)  
        if  
(search_2025(node_2025.getRight_2025(), data_2025))  
            return true;  
    return false;  
}
```

Kode Program 2.14 *Method Search* Rekursif

Method ini digunakan untuk melakukan pencarian data secara rekursif pada seluruh *node Binary Tree*.

6. Membuat *method traversal tree*

```
public void printInorder_2025() {  
    root_2025.printInorder_2025(root_2025);  
}  
public void printPreorder_2025() {  
    root_2025.printPreorder_2025(root_2025);  
}  
public void printPostorder_2025() {  
    root_2025.printPostorder_2025(root_2025);  
}
```

Kode Program 2.15 *Method Traversal*

Method ini digunakan untuk menampilkan isi *Binary Tree* menggunakan *traversal Inorder, Preorder, dan Postorder*.

7. Membuat *method getRoot_2025()*

```
public Node_2511532025 getRoot_2025() {  
    return root_2025;  
}
```

Kode Program 2.16 *Method Get Root*

Method ini digunakan untuk mengambil *node root* pada *Binary Tree*.

8. Membuat *method isEmpty_2025*

```
public boolean isEmpty_2025() {  
    return root_2025 == null;  
}
```

Kode Program 2.17 *Method Print Array*

Method ini digunakan untuk menampilkan seluruh isi *array* ke layar.

9. Membuat *method countNodes_2025()*

```
public int countNodes_2025() {  
    return countNodes_2025(root_2025);  
}
```

```

    }
    private int countNodes_2025(Node_2511532025
node_2025) {
        int count_2025 = 1;
        if (node_2025 == null) {
            return 0;
        } else {
            count_2025 +=
countNodes_2025(node_2025.getLeft_2025());
            count_2025 +=
countNodes_2025(node_2025.getRight_2025());
            return count_2025;
        }
    }
}

```

Kode Program 2.18 *Method Count Nodes*

Method ini digunakan untuk menghitung jumlah seluruh *node* yang terdapat pada *Binary Tree*.

10. Membuat *method print_2025()*

```

public void print_2025() {
    root_2025.print_2025();
}

```

Kode Program 2.19 *Method Print Tree*

Method ini digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk diagram.

11. Membuat *method getCurrent_2025()* dan *setCurrent_2025()*

```

public Node_2511532025 getCurrent_2025() {
    return currentNode_2025;
}
public void setCurrent_2025(Node_2511532025
node_2025) {
    this.currentNode_2025 = node_2025;
}

```

Kode Program 2.20 *Method Get dan Set Current Node*

Method ini digunakan untuk mengambil dan mengubah *node* yang sedang aktif.

12. Membuat *method setRoot_2025()*

```
public void setRoot_2025(Node_2511532025 root_2025)
{
    this.root_2025 = root_2025;
}
```

Kode Program 2.21 *Method Set Root*

Method ini digunakan untuk mengatur *node* yang akan dijadikan *root* pada *Binary Tree*.

2.3 Program *BtreeDriver*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan9_2511532025;

public class BtreeDriver_2511532025 {
    public static void main(String[] args) {
        // Membuat Pohon
        BTree_2511532025 tree_2025 = new BTree_2511532025();
        System.out.print("Jumlah simpul awal pohon: ");
        System.out.println(tree_2025.countNodes_2025());
        // Menambahkan simpul data 1
        Node_2511532025 root_2025 = new Node_2511532025(1);
        // Menjadikan simpul 1 sebagai root
        tree_2025.setRoot_2025(root_2025);
        System.out.println("Jumlah simpul jika hanya ada
root");
        System.out.println(tree_2025.countNodes_2025());
        Node_2511532025 node2_2025 = new Node_2511532025(2);
        Node_2511532025 node3_2025 = new Node_2511532025(3);
        Node_2511532025 node4_2025 = new Node_2511532025(4);
        Node_2511532025 node5_2025 = new Node_2511532025(5);
        Node_2511532025 node6_2025 = new Node_2511532025(6);
        Node_2511532025 node7_2025 = new Node_2511532025(7);
        Node_2511532025 node8_2025 = new Node_2511532025(8);
        Node_2511532025 node9_2025 = new Node_2511532025(9);
        root_2025.setLeft_2025(node2_2025);
        node2_2025.setLeft_2025(node4_2025);
        node2_2025.setRight_2025(node5_2025);
        node4_2025.setRight_2025(node8_2025);
        root_2025.setRight_2025(node3_2025);
        node3_2025.setLeft_2025(node6_2025);
        node3_2025.setRight_2025(node7_2025);
        node6_2025.setLeft_2025(node9_2025);
    }
}
```

```

        // Set root
        tree_2025.setCurrent_2025(tree_2025.getRoot_2025());
        System.out.println("menampilkan simpul terakhir: ");

        System.out.println(tree_2025.getCurrent_2025().getData_2025(
));
        System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
        System.out.println(tree_2025.countNodes_2025());
        System.out.println("InOrder: ");
        tree_2025.printInorder_2025();
        System.out.println("\nPreorder: ");
        tree_2025.printPostorder_2025();
        System.out.println("\nMenampilkan simpul dalam
bentuk pohon");
        tree_2025.print_2025();
    }
}

```

Kode Program 2.22 Program *BtreeDriver*

Kode program di atas digunakan untuk menguji implementasi *Binary Tree* yang telah dibuat pada *class Node_2511532025* dan *BTree_2511532025*. Program diawali dengan membuat objek *Binary Tree* yang masih kosong, kemudian membuat beberapa *node* dan menghubungkannya sehingga membentuk sebuah struktur pohon.

Node dengan nilai 1 dijadikan sebagai *root*, kemudian ditambahkan *node-node* lain sebagai anak kiri dan anak kanan sesuai struktur *Binary Tree* yang diinginkan. Setelah *tree* terbentuk, program menghitung jumlah simpul, menampilkan *root* yang sedang aktif, melakukan *traversal Inorder* dan *Postorder*, serta menampilkan struktur *tree* dalam bentuk diagram.

Melalui program ini dapat diketahui bahwa *Binary Tree* berhasil dibentuk dan seluruh *method* yang terdapat pada *class BTree_2511532025* dapat dijalankan dengan baik.

2.3.1 Langkah Kerja Program *BtreeDriver*

1. Membuat *class BtreeDriver_2511532025*

```
public class BtreeDriver_2511532025 {
```

Kode Program 2.23 Menentukan *Pivot*

Class ini digunakan untuk menjalankan dan menguji operasi *Binary Tree*.

2. Membuat *method main()*

```
public static void main(String[] args) {
```

Kode Program 2.24 *Method Main*

Method main() merupakan *method* utama yang digunakan untuk menjalankan program.

3. Membuat objek *Binary Tree*

```
// Membuat Pohon  
BTree_2511532025 tree_2025 = new  
BTree_2511532025();
```

Kode Program 2.25 Membuat Objek *Tree*

Syntax ini digunakan untuk membuat sebuah *Binary Tree* baru.

4. Menampilkan jumlah simpul awal

```
System.out.print("Jumlah simpul awal pohon:  
");  
System.out.println(tree_2025.countNodes_2025());
```

Kode Program 2.26 Menampilkan Jumlah Simpul Awal

Syntax ini digunakan untuk menampilkan jumlah simpul pada *tree* yang masih kosong.

5. Membuat *root*

```
// Menambahkan simpul data 1
Node_2511532025 root_2025 = new
Node_2511532025(1);
```

Kode Program 2.27 Membuat *Root*

Syntax ini digunakan untuk membuat node dengan nilai 1 yang akan dijadikan *root*.

6. Menjadikan *node* sebagai *root*

```
// Menjadikan simpul 1 sebagai root
tree_2025.setRoot_2025(root_2025);
System.out.println("Jumlah simpul jika hanya
ada root");

System.out.println(tree_2025.countNodes_2025());
```

Kode Program 2.28 *Set Root*

Syntax ini digunakan untuk menetapkan *node* 1 sebagai *root Binary Tree*.

7. Membuat *node-node* baru

```
Node_2511532025 node2_2025 = new
Node_2511532025(2);
Node_2511532025 node3_2025 = new
Node_2511532025(3);
Node_2511532025 node4_2025 = new
Node_2511532025(4);
Node_2511532025 node5_2025 = new
Node_2511532025(5);
Node_2511532025 node6_2025 = new
Node_2511532025(6);
Node_2511532025 node7_2025 = new
Node_2511532025(7);
Node_2511532025 node8_2025 = new
Node_2511532025(8);
Node_2511532025 node9_2025 = new
Node_2511532025(9);
```

Kode Program 2.29 Membuat *Node*

Syntax ini digunakan untuk membuat *node-node* yang akan ditambahkan ke dalam *Binary Tree*.

8. Menghubungkan *node-node* menjadi *Binary Tree*

```
root_2025.setLeft_2025(node2_2025);
node2_2025.setLeft_2025(node4_2025);
node2_2025.setRight_2025(node5_2025);
node4_2025.setRight_2025(node8_2025);
root_2025.setRight_2025(node3_2025);
node3_2025.setLeft_2025(node6_2025);
node3_2025.setRight_2025(node7_2025);
node6_2025.setLeft_2025(node9_2025);
```

Kode Program 2.30 Menghubungkan *Node*

Syntax ini digunakan untuk membentuk hubungan antara *parent* dan anak sehingga terbentuk struktur *Binary Tree*.

9. Mengatur *current node*

```
// Set root
tree_2025.setCurrent_2025(tree_2025.getRoot_2025());
System.out.println("menampilkan simpul
terakhir: ");
```

Kode Program 2.31 *Set Current Node*

Syntax ini digunakan untuk menjadikan *root* sebagai *node* yang sedang aktif.

10. Menampilkan data *current node*

```
System.out.println(tree_2025.getCurrent_2025().getDa
ta_2025());
System.out.println("Jumlah simpul; setelah
simpul 7 ditambahkan");
```

Kode Program 2.32 Menampilkan *Current Node*

Syntax ini digunakan untuk menampilkan nilai *node* yang sedang aktif.

11. Menghitung jumlah simpul

```
System.out.println(tree_2025.countNodes_2025());
```

Kode Program 2.33 Menghitung Simpul

Syntax ini digunakan untuk menampilkan jumlah seluruh simpul pada *Binary Tree*.

12. Menampilkan *traversal Inorder*

```
System.out.println("InOrder: ");  
tree_2025.printInorder_2025();
```

Kode Program 2.34 *Traversal Inorder*

Syntax ini digunakan untuk menampilkan *node* dengan urutan *Left → Root → Right*.

13. Menampilkan *traversal Postorder*

```
System.out.println("\nPreorder: ");  
tree_2025.printPostorder_2025();
```

Kode Program 2.35 *Traversal Postorder*

Syntax ini digunakan untuk menampilkan *node* dengan urutan *Left → Right → Root*

14. Menampilkan struktur *Binary Tree*

```
System.out.println("\nMenampilkan simpul  
dalam bentuk pohon");  
tree_2025.print_2025();  
}  
}  
}
```

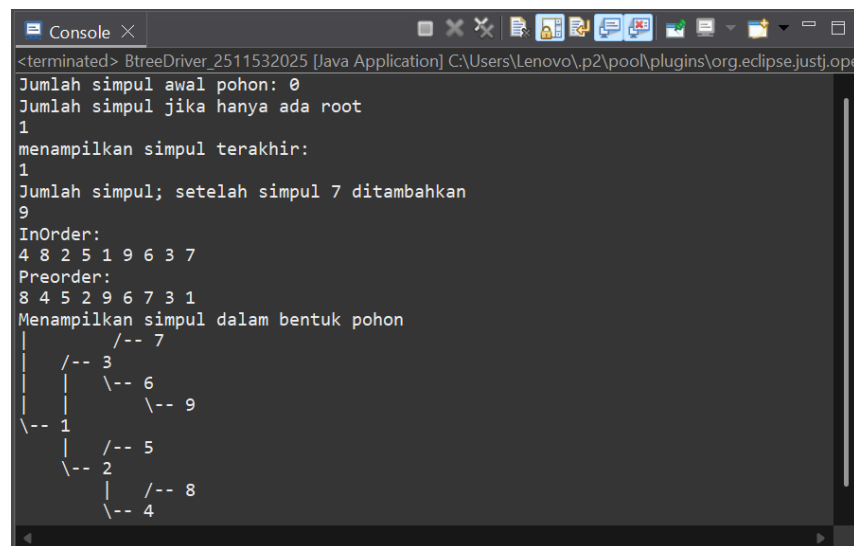
Kode Program 2.36 Menampilkan *Tree*

Syntax ini digunakan untuk menampilkan bentuk *Binary Tree* dalam format diagram.

2.3.2 Hasil Program *BtreeDriver*

Berdasarkan hasil eksekusi program, *Binary Tree* berhasil dibentuk dengan *root* bernilai 1 dan memiliki total 9 simpul. Program dapat menampilkan jumlah simpul sebelum dan sesudah penambahan *node*, menampilkan data pada *current node*, melakukan *traversal Inorder* dan *Postorder*, serta menampilkan struktur pohon secara visual.

Struktur *Binary Tree* yang terbentuk adalah sebagai berikut:



```
<terminated> BtreeDriver_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.j2se-17.0.2-jre\bin\java.exe
Jumlah simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
| /-- 7
| |
| | /-- 3
| | |
| | | \-- 6
| | | \-- 9
| | |
| | | \-- 1
| | | |
| | | | /-- 5
| | | | \-- 2
| | | | |
| | | | | /-- 8
| | | | | \-- 4
```

Gambar 2.1 Hasil Program *BtreeDriver*

Hasil tersebut menunjukkan bahwa *node-node* telah terhubung dengan benar dan seluruh operasi pada *Binary Tree* dapat berjalan sesuai dengan yang diharapkan.

2.4 Program *GraphTraversal*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan9_2511532025;
import java.util.*;

public class GraphTraversal_2511532025 {
    private Map<String, List<String>> graph_2025 = new
    HashMap<>();
```

```

// Menambahkan edge (graf tak berarah)
public void addEdge_2025(String node1_2025, String
node2_2025) {
    graph_2025.putIfAbsent(node1_2025, new
ArrayList<>());
    graph_2025.putIfAbsent(node2_2025, new
ArrayList<>());
    graph_2025.get(node1_2025).add(node2_2025);
    graph_2025.get(node2_2025).add(node1_2025);
}
// Menampilkan graf awal
public void printGraph_2025() {
    System.out.println("Graf Awal (Adjacency List):");
    for (String node_2025 : graph_2025.keySet()) {
        System.out.print(node_2025 + " -> ");
        List<String> neighbors_2025 =
graph_2025.get(node_2025);
        System.out.println(String.join(", ",
neighbors_2025));
    }
    System.out.println();
}
// DFS rekursif
public void dfs_2025(String start_2025) {
    Set<String> visited_2025 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_2025(start_2025, visited_2025);
    System.out.println();
}
private void dfsHelper_2025(String current_2025,
Set<String> visited_2025) {
    if (visited_2025.contains(current_2025)) return;
    visited_2025.add(current_2025);
    System.out.print(current_2025 + " ");
    for (String neighbor_2025 :
graph_2025.getDefault(current_2025, new ArrayList<>())) {
        dfsHelper_2025(neighbor_2025, visited_2025);
    }
}
// BFS iteratif
public void bfs_2025(String start_2025) {
    Set<String> visited_2025 = new HashSet<>();
    Queue<String> queue_2025 = new LinkedList<>();
    queue_2025.add(start_2025);
    visited_2025.add(start_2025);
    System.out.println("Penelusuran BFS:");
    while (!queue_2025.isEmpty()) {
        String current_2025 = queue_2025.poll();
        System.out.print(current_2025 + " ");
        for (String neighbor_2025 :
graph_2025.getDefault(current_2025, new ArrayList<>())) {
            if
(!visited_2025.contains(neighbor_2025)) {
                queue_2025.add(neighbor_2025);
                visited_2025.add(neighbor_2025);
            }
        }
    }
}

```



```

    }
    System.out.println();
}

// Main
public static void main(String[] args) {
    GraphTraversal_2511532025 graph_2025 = new
    GraphTraversal_2511532025();

    // Contoh graf: A-B, A-C, B-D, B-E
    graph_2025.addEdge_2025("A", "B");
    graph_2025.addEdge_2025("A", "C");
    graph_2025.addEdge_2025("B", "D");
    graph_2025.addEdge_2025("B", "E");

    // cetak grap awal
    System.out.println("Graf Awal adalah: ");
    graph_2025.printGraph_2025();

    graph_2025.dfs_2025("A");
    graph_2025.bfs_2025("A");

}
}

```

Kode Program 2.37 Program *GraphTraversal*

Kode program di atas digunakan untuk membuat graf tak berarah (*undirected graph*) serta melakukan penelusuran menggunakan algoritma *Depth First Search (DFS)* dan *Breadth First Search (BFS)*. Graf disimpan menggunakan struktur data *Adjacency List* dengan memanfaatkan *Map<String, List<String>>*.

Method addEdge_2025() digunakan untuk menambahkan hubungan antar simpul. *Method printGraph_2025()* digunakan untuk menampilkan isi graf. *Method dfs_2025()* digunakan untuk melakukan penelusuran secara mendalam menggunakan rekursi, sedangkan *method bfs_2025()* digunakan untuk melakukan penelusuran per level menggunakan *queue*.

Melalui program ini dapat dipahami cara kerja algoritma *DFS* dan *BFS* dalam menelusuri simpul-simpul pada sebuah graf.

2.4.1 Langkah Kerja Program *GraphTraversal*

1. Membuat *class GraphTraversal_2511532025*

```
public class GraphTraversal_2511532025 {
```

Kode Program 2.38 *Class GraphTraversal*

Class ini digunakan untuk membuat dan mengelola graf.

2. Membuat *adjacency list*

```
private Map<String, List<String>> graph_2025 = new  
HashMap<>();
```

Kode Program 2.39 *Adjacency List*

Variabel ini digunakan untuk menyimpan hubungan antar simpul dalam graf.

3. Membuat *method addEdge_2025()*

```
// Menambahkan edge (graf tak berarah)  
public void addEdge_2025(String node1_2025, String  
node2_2025) {  
    graph_2025.putIfAbsent(node1_2025, new  
ArrayList<>());  
    graph_2025.putIfAbsent(node2_2025, new  
ArrayList<>());  
    graph_2025.get(node1_2025).add(node2_2025);  
    graph_2025.get(node2_2025).add(node1_2025);  
}
```

Kode Program 2.40 *Method Add Edge*

Method ini digunakan untuk menambahkan sisi (*edge*) antara dua simpul pada graf.

4. Membuat *method printGraph_2025()*

```
// Menampilkan graf awal  
public void printGraph_2025() {  
    System.out.println("Graf Awal (Adjacency  
List):");  
    for (String node_2025 : graph_2025.keySet())  
{  
        System.out.print(node_2025 + " -> ");  
    }
```

```

        List<String> neighbors_2025 =
graph_2025.get(node_2025);
        System.out.println(String.join(", ",
neighbors_2025));
    }
    System.out.println();
}

```

Kode Program 2.41 *Method Print Graph*

Method ini digunakan untuk menampilkan seluruh isi graf dalam bentuk *adjacency list*.

5. Membuat *method dfs_2025()*

```

// DFS rekursif
public void dfs_2025(String start_2025) {
    Set<String> visited_2025 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_2025(start_2025, visited_2025);
    System.out.println();
}

```

Kode Program 2.42 *Method DFS*

Method ini digunakan untuk memulai penelusuran graf menggunakan algoritma *Depth First Search (DFS)*.

6. Membuat *method dfsHelper_2025()*

```

private void dfsHelper_2025(String current_2025,
Set<String> visited_2025) {
    if (visited_2025.contains(current_2025))
return;
    visited_2025.add(current_2025);
    System.out.print(current_2025 + " ");
    for (String neighbor_2025 :
graph_2025.getOrDefault(current_2025, new
ArrayList<>())) {
        dfsHelper_2025(neighbor_2025,
visited_2025);
    }
}

```

Kode Program 2.43 *Method DFS Rekursif*

Method ini digunakan untuk melakukan penelusuran *DFS* secara rekursif hingga seluruh simpul yang terhubung dikunjungi.

7. Membuat *method bfs_2025()*

```
// BFS iteratif
public void bfs_2025(String start_2025) {
    Set<String> visited_2025 = new HashSet<>();
    Queue<String> queue_2025 = new
LinkedList<>();
    queue_2025.add(start_2025);
    visited_2025.add(start_2025);
    System.out.println("Penelusuran BFS:");
    while (!queue_2025.isEmpty()) {
        String current_2025 =
queue_2025.poll();
        System.out.print(current_2025 + " ");
        for (String neighbor_2025 :
graph_2025.getDefault(current_2025, new
ArrayList<>())) {
            if
(!visited_2025.contains(neighbor_2025)) {
                queue_2025.add(neighbor_2025);
                visited_2025.add(neighbor_2025);
            }
        }
        System.out.println();
    }
}
```

Kode Program 2.44 *Method BFS*

Method ini digunakan untuk melakukan penelusuran graf menggunakan algoritma *Breadth First Search (BFS)*.

8. Membuat objek graf

```
// Main
public static void main(String[] args) {
    GraphTraversal_2511532025 graph_2025 = new
GraphTraversal_2511532025();
}
```

Kode Program 2.45 Membuat Objek Graf

Syntax ini digunakan untuk membuat objek graf yang akan digunakan pada program.

9. Menambahkan *edge* pada graf

```
// Contoh graf: A-B, A-C, B-D, B-E
graph_2025.addEdge_2025("A", "B");
graph_2025.addEdge_2025("A", "C");
graph_2025.addEdge_2025("B", "D");
graph_2025.addEdge_2025("B", "E");
```

Kode Program 2.46 Menambahkan *Edge*

Syntax ini digunakan untuk membentuk hubungan antar simpul pada graf.

10. Menampilkan graf

```
// cetak graf awal
System.out.println("Graf Awal adalah: ");
graph_2025.printGraph_2025();
```

Kode Program 2.47 Menampilkan Graf

Syntax ini digunakan untuk menampilkan graf dalam bentuk *adjacency list*.

11. Menjalankan *DFS*

```
graph_2025.dfs_2025("A");
```

Kode Program 2.48 Menjalankan *DFS*

Syntax ini digunakan untuk melakukan penelusuran *DFS* dimulai dari simpul A.

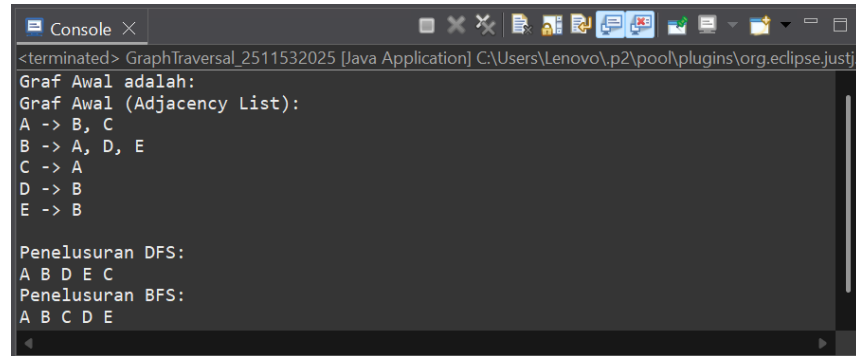
12. Menjalankan *BFS*

```
graph_2025.bfs_2025("A");
}
}
```

Kode Program 2.49 Menjalankan *BFS*

Syntax ini digunakan untuk melakukan penelusuran *BFS* dimulai dari simpul A.

2.4.2 Hasil Program *GraphTraversal*



```
<terminated> GraphTraversal_2511532025 [Java Application] C:\Users\Lenovo\p2\pool\plugins\org.eclipse.justi.  
Graf Awal adalah:  
Graf Awal (Adjacency List):  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```

Gambar 2.2 Hasil Program *GraphTraversal*

Berdasarkan hasil eksekusi program, graf berhasil dibentuk menggunakan metode *adjacency list*. Program menampilkan hubungan antar simpul yang terdiri dari simpul A, B, C, D, dan E. Setelah graf ditampilkan, program melakukan penelusuran menggunakan algoritma *DFS* (*Depth First Search*) dan *BFS* (*Breadth First Search*) yang dimulai dari simpul A.

Pada proses *DFS*, penelusuran dilakukan dengan mengunjungi simpul sedalam mungkin terlebih dahulu sebelum berpindah ke cabang lainnya. Sedangkan pada proses *BFS*, penelusuran dilakukan secara bertahap berdasarkan tingkat kedekatan simpul terhadap simpul awal. Hasil yang ditampilkan menunjukkan bahwa kedua algoritma berhasil menelusuri seluruh simpul pada graf sesuai dengan karakteristik masing-masing metode.

BAB III

KESIMPULAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *Binary Tree* dan *Graph* merupakan struktur data non-linear yang digunakan untuk menyimpan serta mengelola data yang memiliki hubungan tertentu antar elemennya. Pada praktikum ini digunakan beberapa *class*, yaitu *Node_2511532025*, *BTree_2511532025*, *BtreeDriver_2511532025*, dan *GraphTraversal_2511532025* yang diimplementasikan menggunakan bahasa Java.

Melalui program *Node_2511532025*, dapat dipahami bahwa *node* merupakan komponen dasar pembentuk *Binary Tree* yang terdiri atas data, anak kiri (*left child*), dan anak kanan (*right child*). Setiap *node* dapat saling terhubung sehingga membentuk struktur pohon.

Pada program *BTree_2511532025*, *Binary Tree* dikelola melalui berbagai operasi seperti pencarian data, perhitungan jumlah simpul, serta *traversal* menggunakan metode *Inorder*, *Preorder*, dan *Postorder*. Program ini juga menyediakan fungsi untuk menampilkan struktur pohon secara visual sehingga hubungan antar simpul dapat terlihat dengan jelas.

Selanjutnya, program *BtreeDriver_2511532025* digunakan untuk menguji implementasi *Binary Tree* dengan membuat beberapa simpul dan menghubungkannya menjadi sebuah pohon. Hasil pengujian menunjukkan bahwa proses pembentukan *tree*, perhitungan jumlah simpul, dan *traversal* dapat berjalan dengan baik sesuai yang diharapkan.

Pada program *GraphTraversal_2511532025*, diterapkan struktur data *Graph* menggunakan metode *adjacency list*. Program ini mengimplementasikan algoritma *Depth First Search (DFS)* dan *Breadth*

First Search (BFS) untuk melakukan penelusuran simpul pada graf. Hasil yang diperoleh menunjukkan bahwa kedua algoritma dapat menelusuri seluruh simpul dengan urutan yang berbeda sesuai karakteristik masing-masing metode.

Dengan demikian, praktikum ini membantu memahami konsep dasar *Binary Tree* dan *Graph*, cara kerja *traversal* pada *tree* maupun *graph*, serta implementasinya menggunakan bahasa Java. Praktikum ini juga meningkatkan pemahaman dalam merancang struktur data non-linear dan menerapkan algoritma penelusuran data secara efektif.

DAFTAR PUSTAKA

- [1] Wahyudi, S.T., M.T., *Struktur Data: Pekan 13 Binary Tree*. Padang: Universitas Andalas, 2025.
- [2] Wahyudi, S.T., M.T., *Struktur Data: Pekan 14 Struktur Data Graf*. Padang: Universitas Andalas, 2025.

Tugas Praktikum Pekan 9: Struktur Data

Topik:

Tema:

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma *BFS* (*Breadth First Search*) dan *DFS* (*Depth First Search*) menggunakan bahasa Java dengan antarmuka *GUI*. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk *Graph*.

2. Kode Program

```
package pekan9_2511532025;

import javax.swing.*;
import java.awt.*;
import java.util.*;

public class PetaBandara_2511532025 extends JFrame {

    private HashMap<String, ArrayList<String>> graph_2025 = new
HashMap<>();

    private JComboBox<String> start_2025;
    private JComboBox<String> goal_2025;

    private JButton bfsButton_2025;
    private JButton dfsButton_2025;
    private JButton resetButton_2025;

    private JTextArea hasil_2025;
    private JTextArea areaGraph_2025;

    private ArrayList<String> visitedNodes_2025 = new
ArrayList<>();
    private ArrayList<String> path_2025 = new ArrayList<>();

    public PetaBandara_2511532025() {

        createGraph_2025();

        setTitle("Pencarian Jalur Menggunakan BFS dan DFS");
        setSize(1000, 500);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
```

```

        JLabel title_2025 = new JLabel("PETA BANDARA",
SwingConstants.CENTER);
        title_2025.setFont(new Font("Arial", Font.BOLD, 22));
        title_2025.setOpaque(true);
        title_2025.setBackground(new Color(25, 55, 109));
        title_2025.setForeground(Color.WHITE);
        title_2025.setPreferredSize(new Dimension(1000, 50));
        add(title_2025, BorderLayout.NORTH);

        JPanel controlPanel_2025 = new JPanel();

        start_2025 = new
JComboBox<>(graph_2025.keySet().toArray(new String[0]));
        goal_2025 = new
JComboBox<>(graph_2025.keySet().toArray(new String[0]));

        bfsButton_2025 = new JButton("BFS");
        dfsButton_2025 = new JButton("DFS");
        resetButton_2025 = new JButton("RESET");

        bfsButton_2025.setBackground(Color.GREEN);
        dfsButton_2025.setBackground(Color.ORANGE);
        resetButton_2025.setBackground(Color.RED);

        controlPanel_2025.add(new JLabel("Start"));
        controlPanel_2025.add(start_2025);
        controlPanel_2025.add(new JLabel("Goal"));
        controlPanel_2025.add(goal_2025);
        controlPanel_2025.add(bfsButton_2025);
        controlPanel_2025.add(dfsButton_2025);
        controlPanel_2025.add(resetButton_2025);

        add(controlPanel_2025, BorderLayout.SOUTH);

        areaGraph_2025 = new JTextArea();
        areaGraph_2025.setEditable(false);
        areaGraph_2025.setFont(new Font("Monospaced", Font.BOLD,
16));
        displayGraph_2025();

        add(new JScrollPane(areaGraph_2025),
BorderLayout.CENTER);

        hasil_2025 = new JTextArea();
        hasil_2025.setEditable(false);
        hasil_2025.setFont(new Font("Monospaced", Font.PLAIN,
14));

        JScrollPane scroll_2025 = new JScrollPane(hasil_2025);
        scroll_2025.setPreferredSize(new Dimension(500, 0));
        add(scroll_2025, BorderLayout.EAST);

        bfsButton_2025.addActionListener(e -> BFS_2025());
        dfsButton_2025.addActionListener(e -> DFS_2025());
        resetButton_2025.addActionListener(e ->
resetGraph_2025());

```



```

private String displayPath_2025() {

    StringBuilder sb = new StringBuilder();

    for (int i = 0; i < path_2025.size(); i++) {
        sb.append(path_2025.get(i));
        if (i < path_2025.size() - 1) sb.append(" -> ");
    }

    return sb.toString();
}

private void BFS_2025() {

    String start = start_2025.getSelectedItemAt().toString();
    String goal = goal_2025.getSelectedItemAt().toString();

    visitedNodes_2025.clear();
    path_2025.clear();

    Queue<String> q = new LinkedList<>();
    HashSet<String> visited = new HashSet<>();
    HashMap<String, String> parent = new HashMap<>();

    q.add(start);
    visited.add(start);

    while (!q.isEmpty()) {

        String cur = q.poll();
        visitedNodes_2025.add(cur);

        if (cur.equals(goal)) break;

        for (String next : graph_2025.get(cur)) {

            if (!visited.contains(next)) {
                visited.add(next);
                parent.put(next, cur);
                q.add(next);
            }
        }
    }

    buildPath_2025(goal, parent);
    tampilkanHasil_2025("BFS");
}

private void DFS_2025() {

    String start = start_2025.getSelectedItemAt().toString();
    String goal = goal_2025.getSelectedItemAt().toString();

    visitedNodes_2025.clear();
    path_2025.clear();

    Stack<String> st = new Stack<>();

```

```

HashSet<String> visited = new HashSet<>();
HashMap<String, String> parent = new HashMap<>();

st.push(start);

while (!st.isEmpty()) {

    String cur = st.pop();

    if (visited.contains(cur)) continue;

    visited.add(cur);
    visitedNodes_2025.add(cur);

    if (cur.equals(goal)) break;

    for (String next : graph_2025.get(cur)) {

        if (!visited.contains(next)) {
            parent.put(next, cur);
            st.push(next);
        }
    }

    buildPath_2025(goal, parent);
    tampilkanHasil_2025("DFS");
}

private void buildPath_2025(String goal, HashMap<String,
String> parent) {

    String step = goal;

    while (step != null) {
        path_2025.add(0, step);
        step = parent.get(step);
    }
}

private void tampilkanHasil_2025(String metode) {

    hasil_2025.setText(
        "Metode: " + metode +
        "\n\nJalur:\n" + displayPath_2025() +
        "\n\nNode Dikunjungi:\n" + visitedNodes_2025 +
        "\n\nJumlah Node: " + visitedNodes_2025.size()
    );
}

private void resetGraph_2025() {

    visitedNodes_2025.clear();
    path_2025.clear();
    hasil_2025.setText("");
    start_2025.setSelectedIndex(0);
    goal_2025.setSelectedIndex(0);
}

```

```

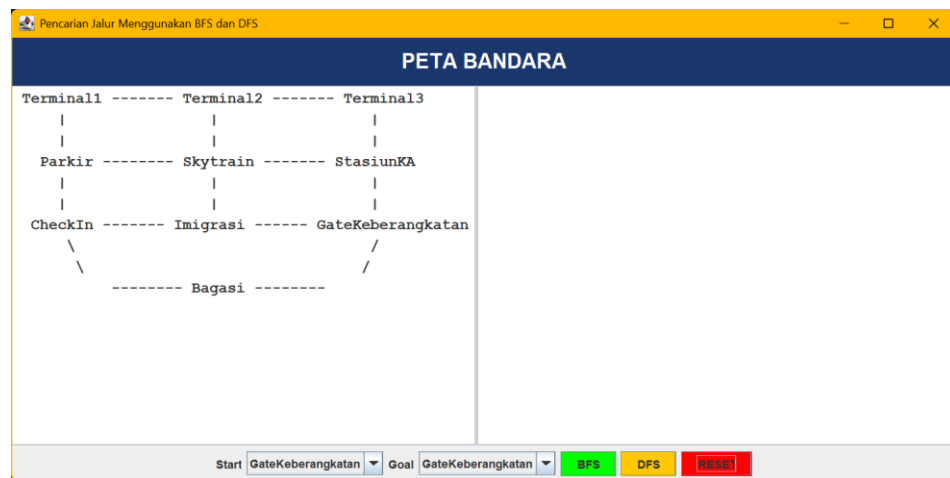
        displayGraph_2025();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new PetaBandara_2511532025());
    }
}

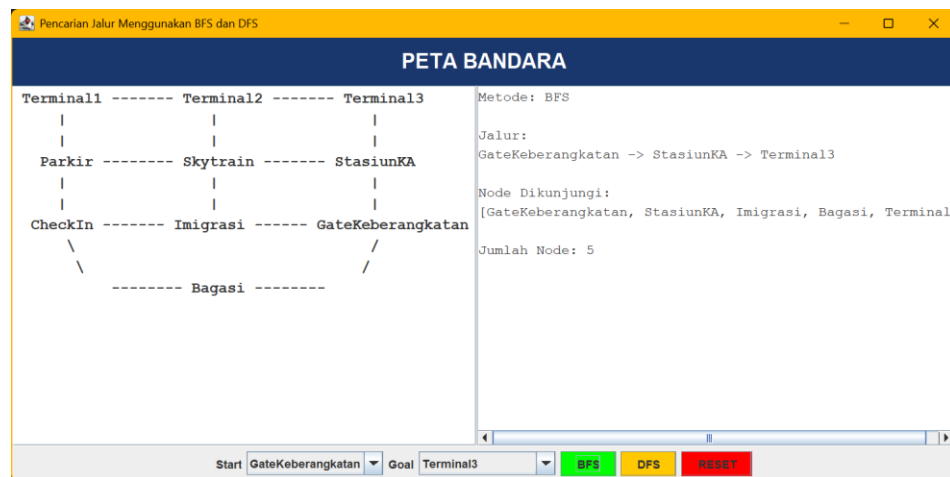
```

3. Tampilan Output Program

Tampilan awal:



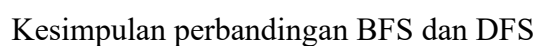
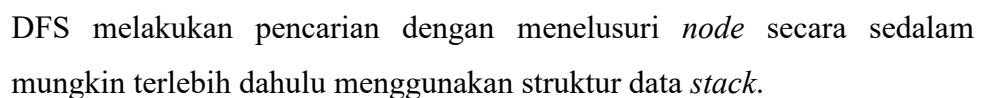
Metode BFS



Metode DFS

Bagasi – Imigrasi

BFS melakukan pencarian dengan menelusuri *node* berdasarkan tingkat kedalaman (level) terlebih dahulu menggunakan struktur data *queue*.



BFS (Breadth First Search) lebih optimal untuk mencari jalur terpendek karena menelusuri *node* berdasarkan level atau kedalaman yang sama terlebih dahulu. Sedangkan *DFS (Depth First Search)* lebih cepat dalam eksplorasi jalur tertentu karena menelusuri satu jalur secara mendalam sebelum kembali.